

Football Betting Model

Testing & Benchmarking Guide

A Step-by-Step Guide for Beginners

Covers: Model Evaluation · Backtesting · Interpreting Results

XGBoost · LightGBM · Ensemble Prediction

English Football | Premier League to National League

Version 1.0 · February 2026

Table of Contents

1. Introduction

- *What this guide covers*
- *How the model works — a plain-English overview*
- *Key concepts you need to know*

2. Setting Up Your Environment

- *Prerequisites & dependencies*
- *Running the project for the first time*
- *Understanding the project folder structure*

3. Step 1 — Downloading Data

- *What data is downloaded*
- *How to run the download step*
- *Verifying the downloaded data*

4. Step 2 — Training the Models

- *What happens during training*
- *How to run the training step*
- *Understanding training output*

5. Step 3 — Evaluating Model Quality

- *Accuracy, Log Loss, and Brier Score explained*
- *Reading the calibration curve*
- *Reading the feature importance chart*
- *Good vs bad numbers — what to look for*

6. Step 4 — Running the Backtest

- *What backtesting is and why it matters*
- *How to run the backtest*
- *Reading the backtest report*
- *Understanding the ROI curve*

7. Interpreting Your Results

- *Complete metrics reference table*
- *What the numbers mean in plain English*
- *Red flags to watch out for*

8. Tweaking the Model (config.yaml)

- *Key settings you can change*
- *Conservative vs aggressive configurations*

9. Common Problems & Fixes

10. Responsible Use & Limitations

1. Introduction

What This Guide Covers

This guide walks you through **testing and benchmarking** the Football Betting Model from start to finish. By the end you will be able to:

- Download 16 seasons of English football match data
- Train three machine-learning models (Logistic Regression, XGBoost, LightGBM)
- Evaluate how *well* those models actually predict match outcomes
- Run a historical simulation (backtest) to see how a betting strategy would have performed
- Read and understand every number in the results
- Safely tweak settings to see how they affect performance

■ **Warning:** Past performance does not guarantee future results. This project is for educational and research purposes. Never bet more than you can afford to lose.

How the Model Works — Plain English

The model learns patterns from thousands of historical football matches. For each upcoming game it estimates three probabilities:

Outcome	What it means	Example
Home Win	The home team wins	Arsenal 2-0 Chelsea
Draw	The match ends in a draw	Arsenal 1-1 Chelsea
Away Win	The away team wins	Arsenal 0-1 Chelsea

It then compares its probability estimates against the bookmaker's implied probabilities (extracted from the betting odds). When the model believes the true probability is meaningfully higher than what the bookmaker implies, that is called a **value bet**.

Key Concepts

Probability: A number between 0 and 1 (or 0–100%) representing how likely something is. 0.6 = 60% chance.

Implied Probability: The probability the bookmaker's odds suggest. Odds of 2.00 imply a 50% chance.

Value Bet: A bet where the model thinks the true probability is higher than the bookmaker's implied probability by at least 10% (configurable).

Overround / Vig: The bookmaker's built-in margin. Odds across all outcomes sum to >100%, the excess is profit for the bookmaker.

Kelly Criterion: A mathematical formula that calculates how much of your bankroll to bet based on your edge and the odds. The model uses a very conservative 1/20th Kelly.

Backtest: Re-running your strategy on historical data to see how it would have performed — like a flight simulator for betting strategies.

Chronological Split: The model is trained on older matches and tested on newer ones to simulate real-world usage and prevent data leakage.

Calibration: How closely the model's predicted probabilities match reality. A well-calibrated model that says '70%' should be right ~70% of the time.

ROI (Return on Investment): Profit expressed as a percentage of total amount staked. ROI of 10% means for every £100 staked, you gained £10 net.

2. Setting Up Your Environment

Prerequisites

Before running anything, make sure the following are installed on your computer:

- **Python 3.9 or higher** — download from python.org
- **pip** — comes bundled with Python
- **Git** (optional) — for cloning the repository
- **4 GB free disk space** — for data and model files

Installing Dependencies

Open a terminal, navigate to the project folder, and run:

```
# Navigate to the project folder
cd /path/to/sportsbetting_model

# (Recommended) Create and activate a virtual environment
python -m venv venv
source venv/bin/activate          # Mac / Linux
venv\Scripts\activate            # Windows

# Install all required libraries
pip install -r requirements.txt
```

■ **Info:** The requirements.txt file lists all necessary libraries (pandas, xgboost, lightgbm, scikit-learn, etc.). The install may take 2–5 minutes on a fresh environment.

Project Folder Structure

Here is what each important file and folder does:

Path	What it is
main.py	Command-line entry point — the main script you will run
config.yaml	All tuneable settings (thresholds, model parameters, odds ranges)
gradio_app.py	Optional graphical web interface (click-based, no terminal needed)
src/data_loader.py	Downloads and merges historical match CSV files
src/feature_engineering.py	Calculates team form, head-to-head stats, rest days, etc.
src/modeling.py	Trains and evaluates Logistic Regression, XGBoost, LightGBM
src/backtest.py	Simulates the betting strategy on historical test data
src/prediction.py	Makes predictions and identifies value bets
data/raw/	Downloaded raw CSV files (created automatically)
data/processed/	Merged & feature-engineered data files

data/visualizations/	Generated charts, reports and bet history CSVs
models/	Saved trained model files (.joblib)

3. Step 1 — Downloading Data

What Data Is Downloaded

The model downloads **16 seasons** (2010/11 – 2025/26) of English football match data from football-data.co.uk across four divisions:

- **E0** — Premier League
- **E2** — League One
- **E3** — League Two
- **EC** — National League (Non-League / Conference)

Each CSV file contains: match date, home team, away team, full-time score, half-time score, and betting odds from multiple bookmakers (Bet365, Pinnacle, etc.).

How to Run the Download

In your terminal (with the virtual environment active) run:

```
python main.py --download
```

You will see progress messages like:

```
[INFO] Downloading E0 2024/25 ...
```

```
[INFO] Downloading E2 2024/25 ...
```

```
[INFO] Merging 64 files ...
```

```
[INFO] Data download complete. 48,320 matches saved to data/processed/all_matches.csv
```

Verifying the Download

After download, quickly verify everything is in order:

```
# Check the file exists and has rows
```

```
python -c "
```

```
import pandas as pd
```

```
df = pd.read_csv('data/processed/all_matches.csv')
```

```
print('Total matches:', len(df))
```

```
print('Date range:', df['Date'].min(), 'to', df['Date'].max())
```

```
print('Divisions:', df['Div'].unique())
```

```
"
```

Expected output (approximately):

```
Total matches: 45000
```

```
Date range: 2010-08-07 to 2026-02-22
```

```
Divisions: ['E0' 'E2' 'E3' 'EC']
```

■ **Tip:** If fewer than 40,000 matches are shown, some files may have failed to download (network issue). Run --download again — it skips files that already exist.

4. Step 2 — Training the Models

What Happens During Training

The training step performs the following operations in order:

- 1 **Load data** — reads the merged CSV from Step 1
- 2 **Feature engineering** — calculates team form (last 5 & 10 games), head-to-head records, rest days, goal differences, and implied probabilities from the bookmaker odds
- 3 **Chronological split** — the most recent 15% of matches become the test set; the rest are for training. This is critical to prevent 'data leakage'
- 4 **Train Baseline** — a simple Logistic Regression model (the benchmark)
- 5 **Train XGBoost** — a powerful gradient-boosting model with 300 trees
- 6 **Train LightGBM** — a faster alternative gradient-boosting model
- 7 **Evaluate each model** — prints Accuracy, Log Loss, and Brier Score
- 8 **Save models** — stored in the models/ folder as .joblib files
- 9 **Generate charts** — calibration curves and feature importance plots

How to Run Training

```
python main.py --train
```

Training takes approximately **3–10 minutes** depending on your hardware. You will see live progress messages in the terminal.

Understanding Training Output

The terminal will print metrics for each model. Here is a sample:

```
[INFO] Train set: 41072 matches (2010-08-07 to 2024-07-31)
[INFO] Test set: 7248 matches (2024-08-01 to 2026-02-22)
[INFO] Prepared 6891 samples with 42 features

[INFO] Baseline (Logistic Regression)
[INFO] Accuracy: 0.4712 Log Loss: 1.0231 Brier: 0.2187

[INFO] XGBoost
[INFO] Accuracy: 0.4835 Log Loss: 0.9881 Brier: 0.2101

[INFO] LightGBM
[INFO] Accuracy: 0.4821 Log Loss: 0.9914 Brier: 0.2108
```

■ **Info:** Don't worry about what these numbers mean yet — Section 5 explains each metric in detail with clear thresholds for 'good' and 'bad'.

Output Files After Training

- **models/baseline_model.joblib** — saved Logistic Regression model

- **models/xgboost_model.joblib** — saved XGBoost model
- **models/lightgbm_model.joblib** — saved LightGBM model
- **data/processed/all_matches_features.csv** — data with engineered features
- **data/visualizations/calibration_xgboost.png** — calibration chart
- **data/visualizations/calibration_lightgbm.png** — calibration chart
- **data/visualizations/feature_importance_xgboost.png** — feature chart
- **data/visualizations/feature_importance_lightgbm.png** — feature chart

5. Step 3 — Evaluating Model Quality

The Three Core Metrics

The model reports three metrics after training. Here is what each one means:

Metric 1: Accuracy

What it is: The percentage of matches where the model correctly predicted the outcome (Home Win, Draw, or Away Win).

Formula: Correct predictions ÷ Total predictions

Example: Accuracy of 0.48 means the model predicted the right outcome in 48 out of 100 matches.

Accuracy Range	What it means	Assessment
< 0.40	Worse than random guessing	Poor — something is wrong
0.40 – 0.46	Below average for football prediction	Needs improvement
0.47 – 0.53	Typical range for football ML models	Good ✓
> 0.53	Exceptional — verify for data leakage	Suspicious — double-check

■ **Info:** Football is inherently unpredictable — even the best models rarely exceed 55% accuracy. An accuracy above 60% almost certainly indicates a bug or data leakage.

Metric 2: Log Loss (Cross-Entropy Loss)

What it is: Measures how confident and correct the model's probability estimates are. Unlike accuracy, it cares about how sure the model was — not just whether it was right or wrong.

Direction: Lower is better. A perfect model would have Log Loss = 0. A model that always predicts 33% for all three outcomes (pure guessing) has Log Loss ≈ 1.099.

Log Loss Range	Assessment
> 1.10	Worse than random — check your data
1.00 – 1.10	About as good as random guessing
0.95 – 1.00	Slightly better than random — acceptable
0.85 – 0.95	Good — model has real predictive power
< 0.85	Excellent — verify for data leakage

Metric 3: Brier Score

What it is: Measures the mean squared error between the model's predicted probabilities and the actual outcomes. Think of it as a 'how wrong were the probabilities' score.

Direction: Lower is better. The worst possible score is 1.0; a random model scores ~0.222 for a 3-class problem.

Brier Score	Assessment
-------------	------------

> 0.25	Worse than random
0.21 – 0.25	About as good as random — needs work
0.18 – 0.21	Good — probabilities are meaningful
< 0.18	Excellent

Reading the Calibration Curve

The calibration curve is saved to **data/visualizations/calibration_xgboost.png**. It shows three charts — one for each outcome (Home Win, Draw, Away Win).

Each chart has a **dashed diagonal line** representing perfect calibration. The model's line (solid with dots) should follow this diagonal closely.

- **Line above the diagonal:** model is *under-confident* — it predicts 40% but actually wins 55% of the time
- **Line below the diagonal:** model is *over-confident* — it predicts 70% but only wins 50% of the time
- **Line on the diagonal:** *perfect calibration* — predicted probabilities match reality

■ **Tip:** For value betting, calibration is more important than raw accuracy. A well-calibrated model's probability estimates can be directly compared to bookmaker odds.

Reading the Feature Importance Chart

The feature importance chart shows which input variables had the most influence on predictions. Open **data/visualizations/feature_importance_xgboost.png**.

Common top features you should expect to see:

- **home_form_5_points** — home team's points in last 5 matches
- **away_form_5_points** — away team's points in last 5 matches
- **true_prob_home_win / true_prob_away_win** — implied probabilities from bookmaker odds (after removing vig)
- **h2h_home_win_rate** — historical head-to-head home win rate
- **home_goal_diff / away_goal_diff** — goal difference over recent form window

■ **Warning:** If odds-based features (true_prob_*) dominate with very high importance, the model is largely following the bookmaker's probabilities. This is normal — bookmakers are very accurate — but it limits the model's edge.

6. Step 4 — Running the Backtest

What Backtesting Is

Backtesting simulates placing bets on the **test set** (the 15% of matches not used for training) using your model's predictions and the actual historical odds. It tells you: 'If you had used this model during this period, what would have happened to your bankroll?'

The simulation applies these rules (set in config.yaml):

- **Only bet when edge > 10%** (model probability – implied probability > 0.10)
- **Only bet within odds range 1.8 – 6.0** (avoids huge favourites and longshots)
- **Stake = 1/20th Kelly** (very conservative — small stakes, low risk)
- **Maximum stake = 5% of starting bankroll per bet** (hard cap on any single bet)
- **Starting bankroll = £1,000**

How to Run the Backtest

```
# Default: uses XGBoost model
```

```
python main.py --backtest
```

```
# To test a different model:
```

```
python main.py --backtest --model lightgbm_model
```

```
python main.py --backtest --model baseline_model
```

■ **Info:** The backtest requires training to have been completed first. If you get an error saying 'engineered features not found', run --train first.

Reading the Backtest Report

The report is printed to the terminal and saved to **data/visualizations/backtest_report.txt**. Here is an example report and what each line means:

```
=====
```

```
BACKTEST REPORT
```

```
=====
```

```
OVERALL METRICS:
```

```
Total Bets:      312
```

```
Total Staked:    $4,820.50
```

```
Total Profit:    $1,417.23
```

```
ROI:             29.40%
```

```
Win Rate:        48.40%
```

```
Average Odds:    2.84
```

```
Final Bankroll:  $2,417.23
```

```
BET TYPE BREAKDOWN:
```

Home Win:

Bets: 148 Profit: \$820.11 ROI: 31.2% Win Rate: 52.0%

Draw:

Bets: 34 Profit: \$112.44 ROI: 18.4% Win Rate: 29.4%

Away Win:

Bets: 130 Profit: \$484.68 ROI: 27.8% Win Rate: 43.8%

What Each Number Means

Field	Plain English Explanation	Target
Total Bets	How many bets met the value threshold	> 100
Total Staked	Total money wagered across all bets	—
Total Profit	Net profit/loss after all bets settled	> 0
ROI	Profit as % of total staked — the key metric	> 5%
Win Rate	% of bets that won — misleading alone, see note	Depends on odds
Average Odds	Mean decimal odds of all bets placed	1.8 – 4.0
Final Bankroll	Starting £1,000 plus/minus net profit	> £1,000

■ **Warning:** Win Rate alone is not a good metric for betting. At average odds of 2.84 you only need a ~35% win rate to break even. The model's 48% win rate at those odds implies genuine edge.

Understanding the ROI Curve

Open [data/visualizations/roi_curve.png](#). It shows two sub-charts:

- **Top chart — Cumulative Profit:** starts at £0 and shows profit/loss after each successive bet. A healthy model trends upward overall with normal drawdown periods (going temporarily below £0).
- **Bottom chart — Bankroll over time:** starts at £1,000. A healthy curve trends upward. Watch for prolonged flat or downward periods (drawdowns).

■ **Tip:** A perfectly smooth upward line would be suspicious — real betting has variance. Expect zigzag patterns with an overall positive trend if the model has genuine edge.

7. Interpreting Your Results

Complete Metrics Quick-Reference

Metric	Good Range	Red Flag	Notes
Accuracy	0.47 – 0.53	> 0.58	Football is hard to predict
Log Loss	0.85 – 0.98	< 0.70	Lower is better
Brier Score	0.18 – 0.21	< 0.15	Lower is better
Backtest ROI	5% – 30%	> 50%	Highly variable season to season
Win Rate	35% – 55%	< 25% or > 65%	Depends heavily on avg odds
Total Bets	> 100	< 50	Too few bets = unreliable stats
Avg Odds	1.8 – 4.5	< 1.5 or > 7.0	Matches config min/max odds

Red Flags — Things to Watch Out For

■ Accuracy > 58% or ROI > 50%

These are unrealistically high results for football betting. It almost certainly means the model accidentally 'saw' future data during training (data leakage). Check that the chronological split is working correctly.

■ Total bets < 50 in the backtest

Too few bets makes all statistics unreliable (high variance). Try lowering the `value_threshold` in `config.yaml` from 0.10 to 0.07.

■ Calibration curve far from diagonal

The model's probabilities are systematically biased. This affects value bet detection. Try re-training with more data.

■ Final bankroll < £500 (lost > 50% starting capital)

The strategy is losing money significantly. Try raising the `value_threshold` (more selective) or using a different model.

■ All bets of the same type (e.g. all Home Win)

The model may have a bias. Check the feature importance chart.

8. Tweaking the Model (config.yaml)

The file **config.yaml** is your control panel. You can change settings without editing any Python code. Always make a backup before changing things: copy config.yaml to config_backup.yaml first.

Key Settings

Setting	Default	What it does	Try lowering to...
backtest.value_threshold	0.10 (10%)	Minimum edge required to place a bet. Higher = fewer but higher-quality bets.	0.07 for more bets
backtest.kelly_fraction	0.05 (1/20 Kelly)	Controls stake size. Lower = smaller stakes, lower risk, lower reward.	0.02 for smaller stakes
backtest.min_odds	1.80	Skip bets below this odds — avoids heavy favourites.	1.5 to include more favourites
backtest.max_odds	6.00	Skip bets above this odds — avoids longshots.	4.0 for more conservative
models.test_size	0.15 (15%)	Proportion of data reserved for testing (not training).	0.20 for larger test set
models.xgboost.n_estimators	300	Number of trees in XGBoost. More = slower training, potentially better model.	500 for experimentation

Conservative vs Aggressive Configurations

```
# --- CONSERVATIVE (fewer bets, higher confidence, lower risk) ---
```

```
backtest:
```

```
  value_threshold: 0.15  # only bet when 15%+ edge
```

```
  kelly_fraction: 0.02  # very small stakes
```

```
  min_odds: 2.00        # skip short-odds favourites
```

```
  max_odds: 5.00        # skip longshots
```

```
# --- AGGRESSIVE (more bets, lower confidence, higher risk) ---
```

```
backtest:
```

```
  value_threshold: 0.07  # bet when 7%+ edge
```

```
  kelly_fraction: 0.10  # larger stakes
```

```
  min_odds: 1.60
```

```
  max_odds: 8.00
```

■ **Warning:** Aggressive settings can amplify both profits AND losses. Always test changes in the backtest simulation before considering any real-money application.

9. Common Problems & Fixes

Problem: ModuleNotFoundError: No module named 'xgboost'

Cause: A required library is not installed.

Fix:

- 1 Run: `pip install -r requirements.txt`
 - 2 Make sure your virtual environment is activated (`source venv/bin/activate`)
-

Problem: FileNotFoundError: data/processed/all_matches.csv not found

Cause: Data hasn't been downloaded yet.

Fix:

- 1 Run: `python main.py --download`
 - 2 Then re-run the failing step
-

Problem: Engineered features not found — run training first

Cause: The backtest is trying to read feature data that hasn't been generated.

Fix:

- 1 Run: `python main.py --train first`
 - 2 Then run: `python main.py --backtest`
-

Problem: Total Bets = 0 in backtest report

Cause: No bets met the value threshold — the model found no edge above 10%.

Fix:

- 1 In `config.yaml`, lower `value_threshold` from 0.10 to 0.07
 - 2 Or check that bookmaker odds columns (B365H, B365D, B365A) exist in your data
-

Problem: Training takes > 30 minutes

Cause: Your computer may be running the feature engineering step on very large data.

Fix:

- 1 This is normal on older hardware — let it run
 - 2 You can reduce `n_estimators` in `config.yaml` from 300 to 100 for faster (but less accurate) training
-

Problem: UnicodeDecodeError when loading CSV

Cause: Some older match CSV files use non-UTF-8 encoding.

Fix:

- 1 Delete the `data/raw/` folder and re-download: `python main.py --download`
 - 2 The data loader handles encoding automatically for most files
-

Problem: Calibration chart shows a flat horizontal line

Cause: The model is outputting nearly identical probabilities for all matches — it has failed to learn.

Fix:

- 1 This usually happens when features are all NaN. Check your data has enough history (run --download first)
 - 2 Re-run with: `python main.py --train`
-

10. Responsible Use & Limitations

Important Limitations

- **Backtesting is not a guarantee.** Historical performance can look very different from live performance due to market changes, line movements, and new bookmaker restrictions.
- **Bookmakers close winning accounts.** Even if the model finds genuine edge, bookmakers may limit or close accounts of consistently profitable bettors.
- **The model uses delayed data.** Team news, injuries, weather, and line-up changes are not captured in historical CSV data.
- **Odds data quality varies.** Some historical odds may be missing or incorrect, which affects backtest reliability.
- **Small sample problem.** Even 300 bets is a relatively small sample in statistics — genuine edge can look like luck over 300 bets, and luck can look like genuine edge.
- **Model drift.** A model trained on 2010–2024 data may become less accurate as team dynamics, playing styles, and league structures change.

Responsible Use Guidelines

This model is a research and educational tool. If you choose to use any insights for real-money betting, please follow these guidelines:

- 1 **Never bet more than you can comfortably afford to lose.** Treat any money used for betting as entertainment expense.
- 2 **Set a fixed monthly budget** and stick to it regardless of wins or losses.
- 3 **Do not chase losses** — a losing run is normal even with a profitable model.
- 4 **Keep records** — track every bet placed, stake, odds, and outcome.
- 5 **Use demo / paper trading first** — run 3–6 months of simulated bets before committing real money.
- 6 **If gambling becomes a problem**, seek help at [BeGambleAware.org](https://www.begambleaware.org) or call 0808 8020 133 (UK, free, 24/7).

Quick Command Cheat Sheet

```
# Step 1: Download data
```

```
python main.py --download
```

```
# Step 2: Train models
```

```
python main.py --train
```

```
# Step 3: Run backtest (XGBoost – default)
```

```
python main.py --backtest
```

```
# Step 3b: Run backtest with LightGBM
```

```
python main.py --backtest --model lightgbm_model
```

```
# Step 4: Run entire pipeline in one command
```

```
python main.py --all
```

```
# Step 5: Use the graphical interface instead
```

```
python gradio_app.py
```

```
# Then open http://localhost:7860 in your browser
```

■ **Tip:** Use `python gradio_app.py` to access a click-based web interface if you prefer not to use the terminal. It offers the same functionality with buttons and progress bars.